



西安电子科技大学  
XIDIAN UNIVERSITY

计算机科学与技术学院  
School of Computer Science and Technology

# 分布式计算

## Distributed Computing

主讲人：李龙海

---

### 第七讲 分布式存储与一致性问题



## 主要内容

- 分布式存储系统要达成的目标
- 两种基本手段：复制、分区(切片)
- 实现复制的基本方法与权衡
- 多副本分布式存储系统的一致性问题
- 实现切片的基本方法与权衡
- 案例：分布式文件系统HDFS



- 理解问题本身比学会如何解决问题更重要（现阶段）。
- 这一部分的参考书：《设计数据密集型应用》第5章~第9章



- **提高数据存储容量**：系统的总存储容量是多个组成节点本地存储容量的聚合，系统总存储容量能够随着组成节点数的增加而增大。（水平可扩展）
- **提高数据吞吐量**：系统的数据读写吞吐量能够随着组成节点数的增加而增大。（水平可扩展）
- **提高可靠性/可用性**：部分存储节点发生故障时数据不丢失；部分存储节点失效时，用户仍然可以读写数据。（容错性）
- **降低数据访问延时**：使用户可以从地理位置上最接近的节点读取数据。（例子：CDN内容分发网络）
- **提高分布式数据处理系统的运行效率。**



- 基本思想：在多个不同的节点上保存相同数据的多个**副本**（ replica）。
- 用途：
  - ① 复制提供了冗余，如果一些节点不可用，剩余的节点仍然可以提供数据服务。
  - ② 多个节点上存储副本也可以提高数据吞吐率、改善访问性能。
- 带来的问题：
  - ① 硬件成本提高。
  - ② 在数据变更时保障多数据副本之间的的一致性是个复杂问题。（需要根据具体应用场景进行权衡）

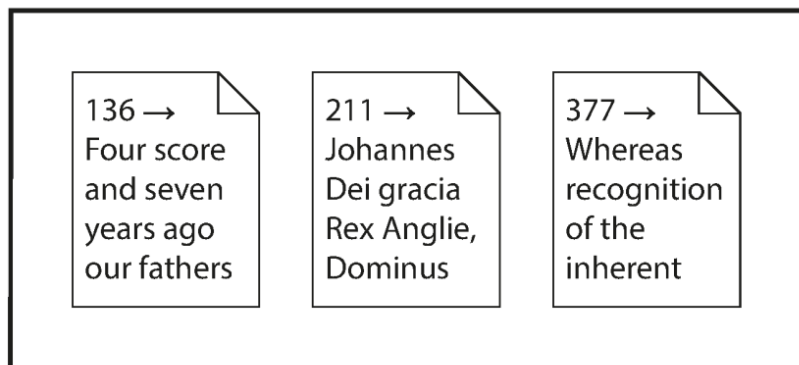


- 基本思想： 将一个大型数据文件(或数据库)拆分成较小的子集（称为分区partition或切片shard），再将不同的分区指派给不同的节点。
- 用途：
  - ① 提高了吞吐率：访问数据的负载被分散到多个节点上。
  - ② 提高了可靠性：鸡蛋被放到多个篮子里
  - ③ 方便了实现数据的并行处理。
- 带来的问题：
  - ① 跨区查询（分布式索引问题）
  - ② 合理、动态分区问题（大数据如何拆分）
  - ③ 负载均衡问题（各个切片如何合理地分配给不同节点）
  - ④ 分布式事务处理

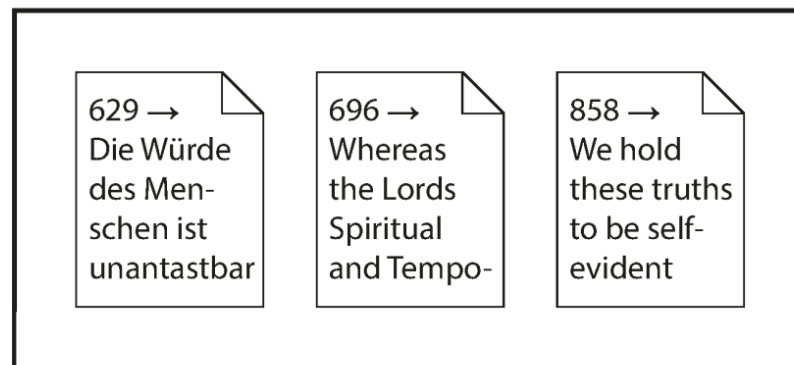


# 数据库被复制和分区的示例

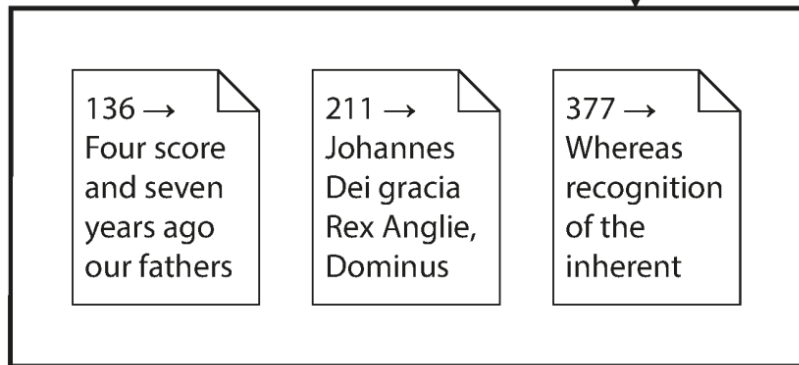
## Partition 1, Replica 1



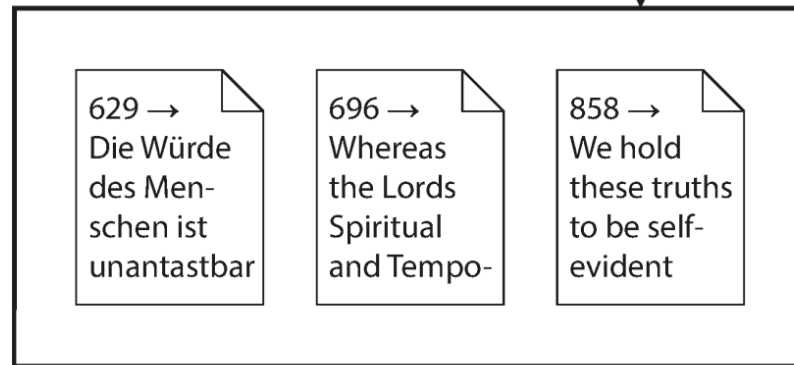
## Partition 2, Replica 1



## Partition 1, Replica 2



## Partition 2, Replica 2



↑  
copy of  
the same  
data  
↓

↑  
copy of  
the same  
data  
↓



- 基于领导者的复制(leader-based replication): 也称为**主动/被动(active/passive)复制**、**主/从(master/slave)复制**, 或**主/备(primary/backup)复制**。
- **基本思想:**
  - ① 副本之一被指定为**领导者(主库)**; 其他副本被指定为**追随者 (从库)**
  - ② 客户端要向数据存储系统写入数据时, 它必须将请求发送给领导者; 领导者将新数据写入本地存储, 同时也会将数据变更发送给所有的追随者。
  - ③ 当客户想要从数据存储系统读取数据时, 它可以向**领导者或追随者**查询。
  - ④ 适合于**读多写少**的应用场景。





User 1234  
configures new  
profile picture



## read-write queries

update users  
set picture\_url = 'me-new.jpg'  
where user\_id = 1234

Leader  
replica



Replication streams

## Data change

table: users  
primary key: 1234  
column: picture\_url  
old\_value: me-old.jpg  
new\_value: me-new.jpg  
transaction: 987654321

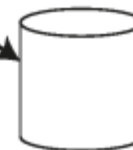
Follower  
replica



## read-only queries

select \* from users  
where user\_id = 1234

Follower  
replica



User 2345  
views user  
1234's profile



## ■ 单主复制：

- ① 一个领导者多个追随者。
- ② 优点：实现相对简单。大多数应用场景都是读多写少。
- ③ 缺点：主库即是性能瓶颈，又是单点故障节点(Single point of failure)
- ④ 为提高容错性，自动将某个从库切换为主库时会面临脑裂问题。

## ■ 多主复制：

- ① 系统中有多个主库接受写入操作。每个主库都将该数据更改转发给所有其他节点。每个领导者同时扮演其他领导者的追随者。
- ② 应用场景：多数据中心之间的复制；协同文档编辑。
- ③ 优点：写入性能高。
- ④ 缺点：写入冲突问题、多副本一致性问题解决方案太复杂。

## ■ 无主复制：

- ① 没有主库从库的概念。
- ② 客户端直接将写入请求发送到各个副本；或者客户端将写入发送给某个节点，由该节点充当代理节点向其他节点转发写入请求。



## ■ 同步复制：

- ① 领导者写入自己的存储器，并且接收到所有跟随者已经成功更新本地存储的应答后再向用户返回成功写入应答。
- ② 优点：更容易实现副本之间的一致性；
- ③ 缺点：写入速度慢；一个节点失效会使整体写入功能失败

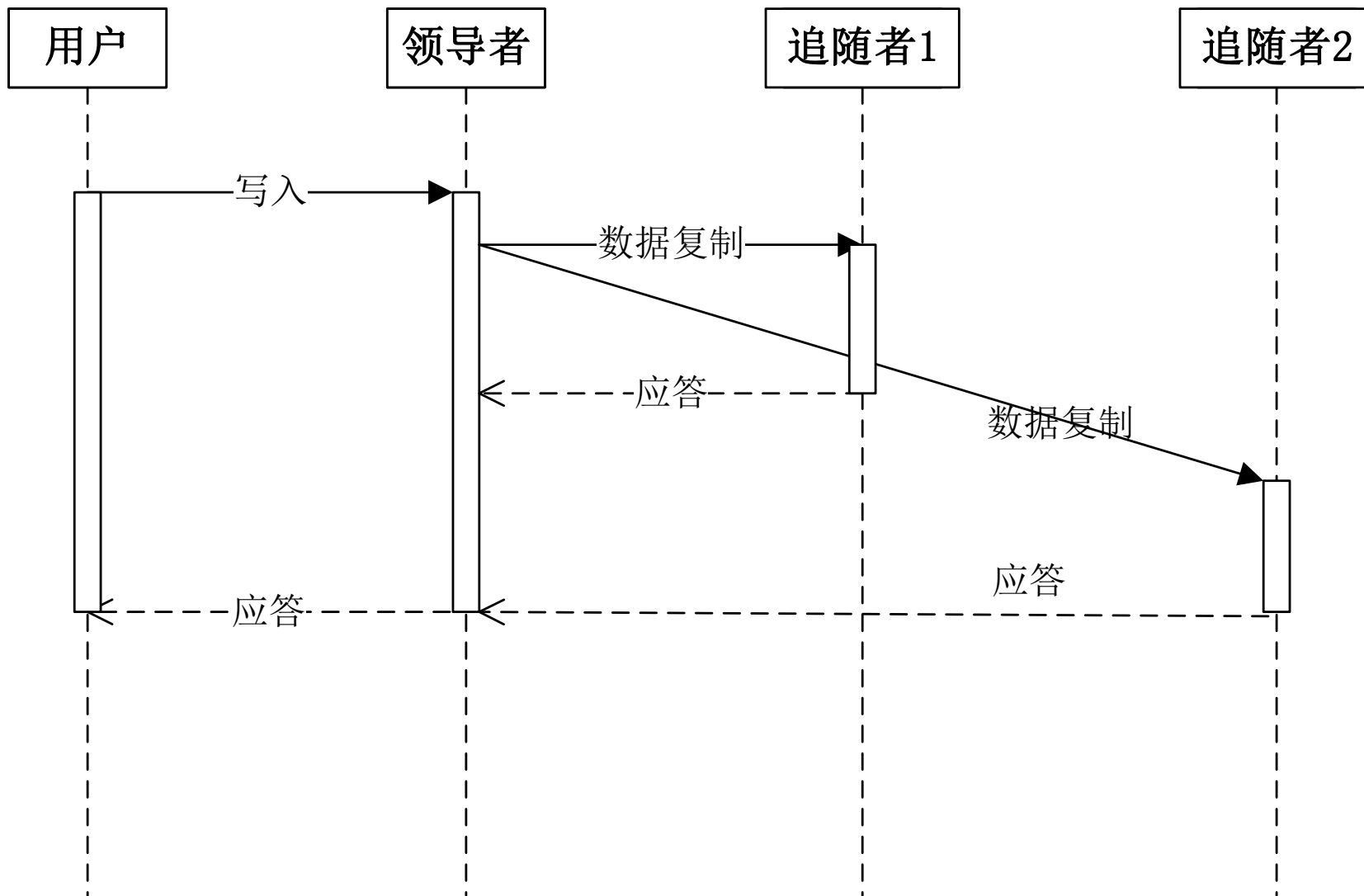
## ■ 异步复制：

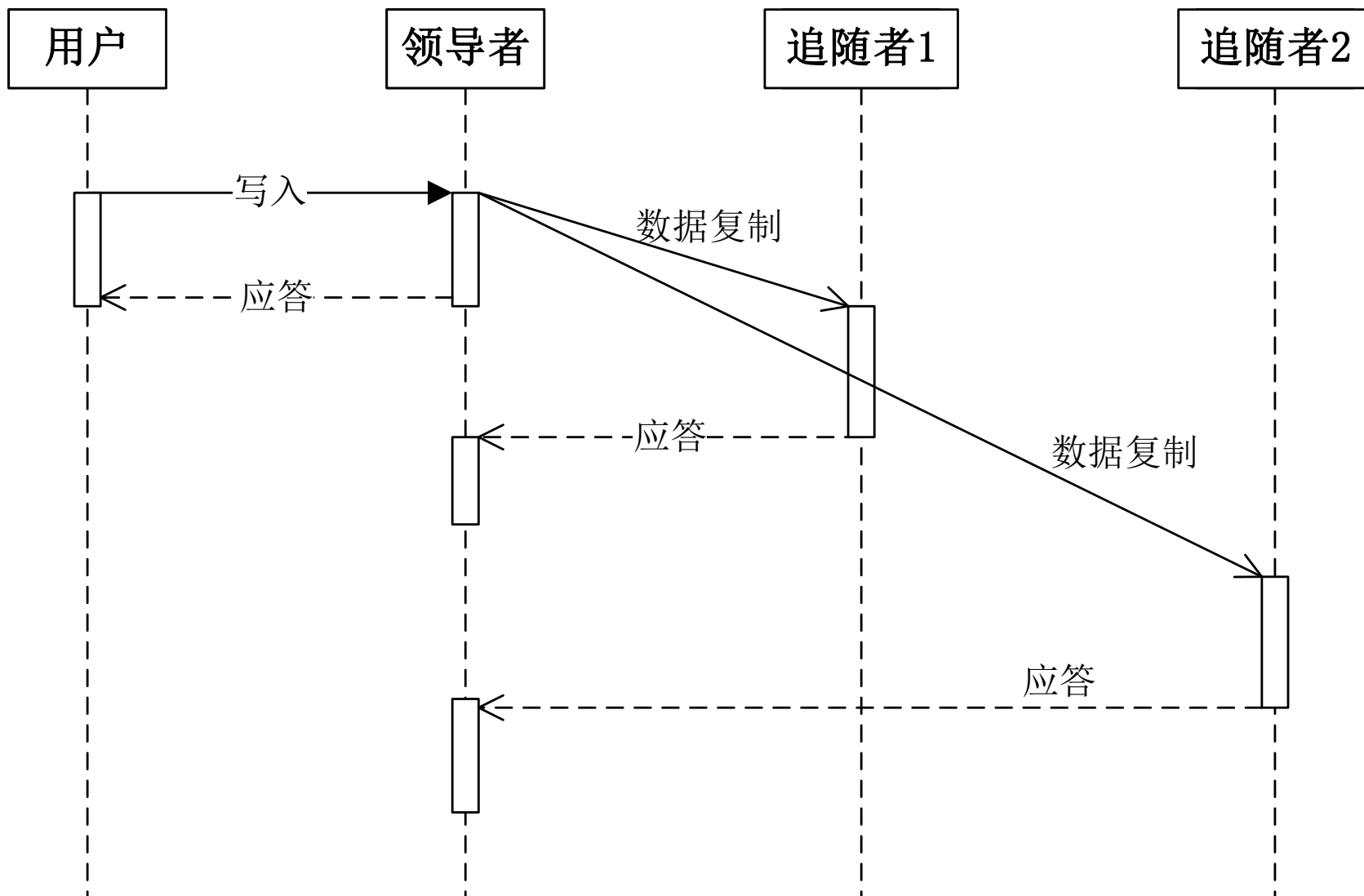
- ① 领导者自己写入成功后立即向用户返回成功应答，不等待其他跟随者的应答消息。
- ② 优点：写入速度快；容错性好；
- ③ 缺点：保持多副本一致性复杂；

## ■ 链式复制：领导者和追随者在复制数据时形成流水线

## ■ 混合复制：

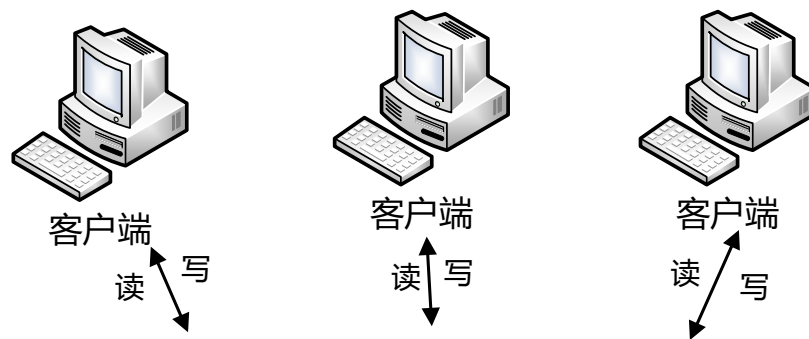
与部分跟随者节点之间采用同步复制，另外的跟随者节点之间采用异步复制。







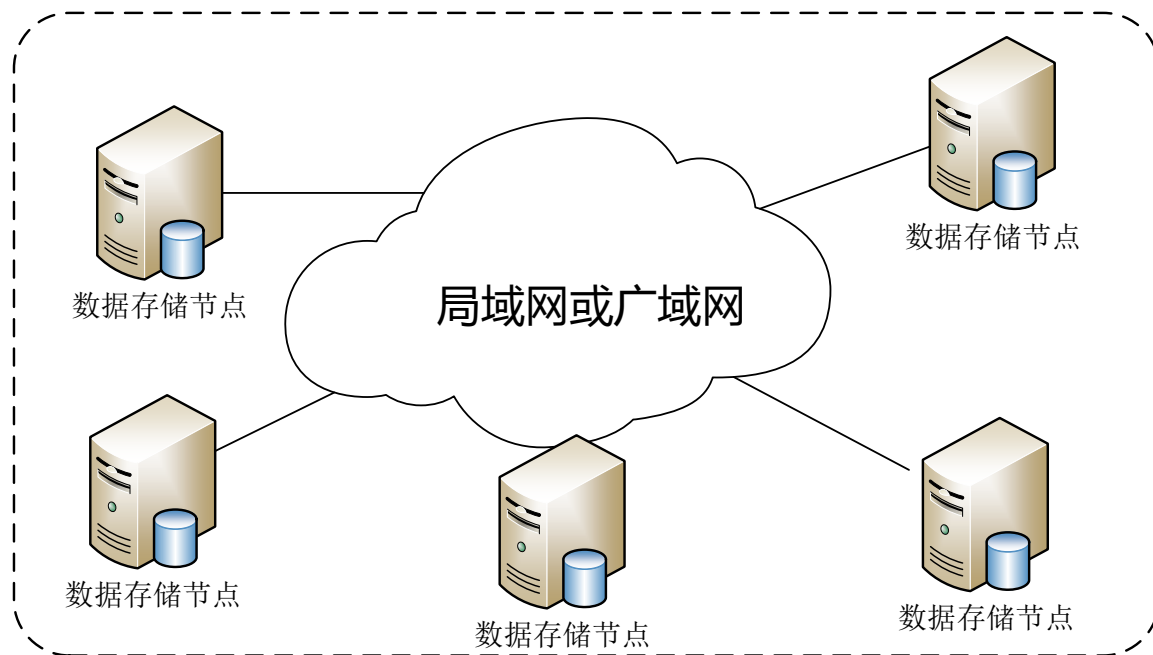
- 多个客户端“同时”对分布式存储系统进行读写。
- 分布式节点之间的网络不可靠：丢包、延迟、断裂、乱序
- 部分存储节点会失效
- 节点的局部时钟不同步



## 数据一致性模型：

多个客户端在读写数据时，分布式存储系统为客户端提供的关于数据外在表现的保证。

客户端和多副本分布式存储系统之间一种服务约定。



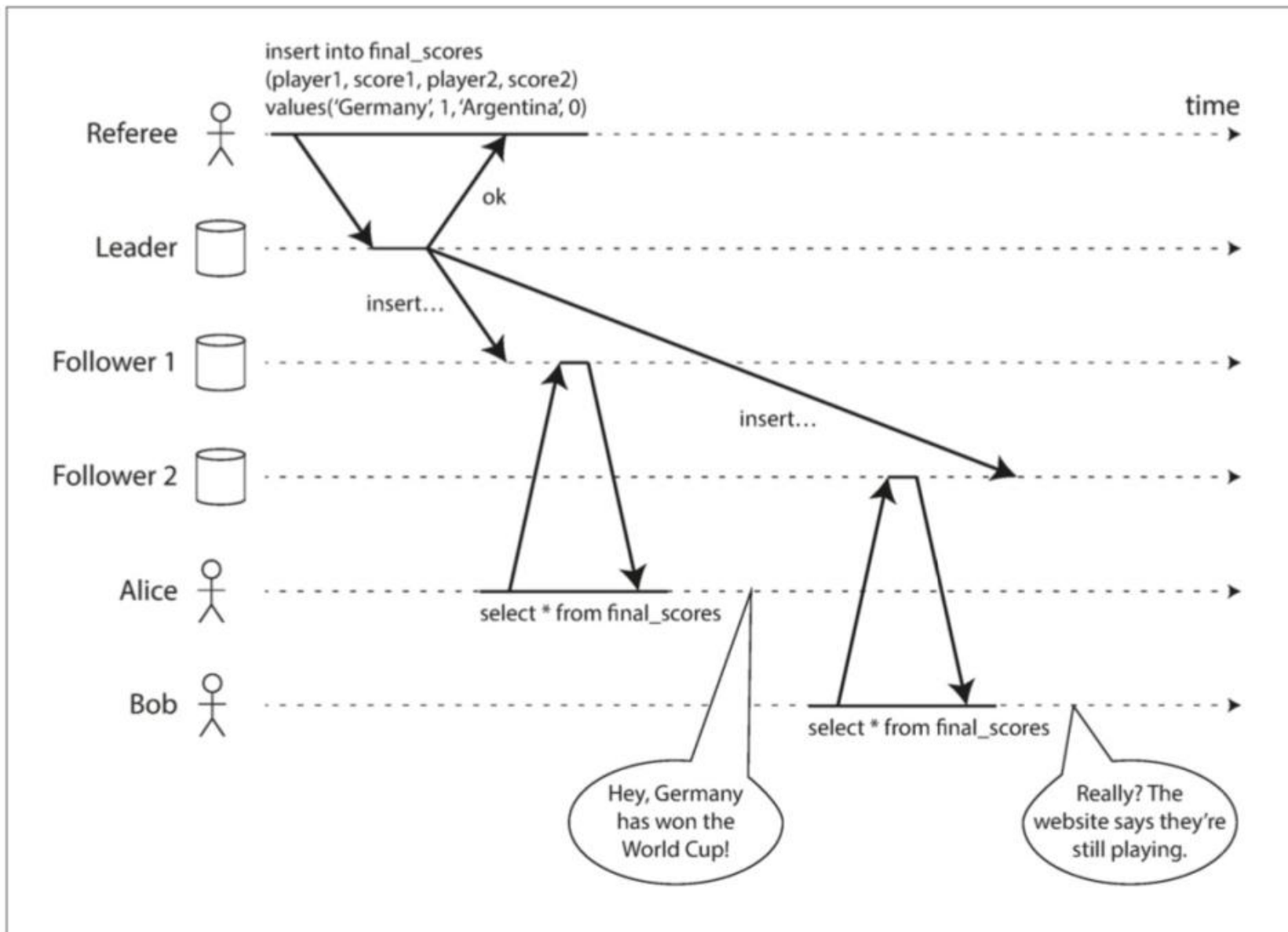


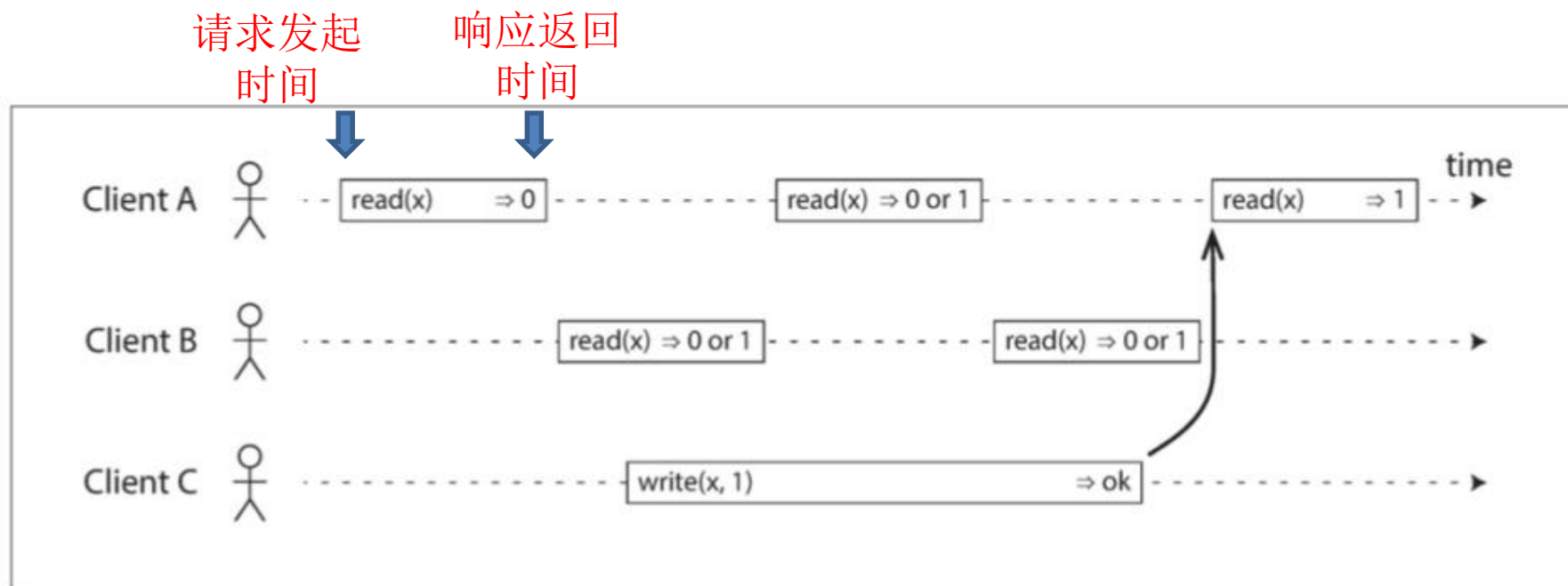
- **强一致性：**（线性一致性/原子一致性）
  - 在客户端看来，分布式存储系统的外在表现和单副本存储系统的外在表现完全一致。（类似于多个线程访问同一个寄存器）
  - 任意客户端看到的所有针对分布式系统的**操作**(读、写等**原子操作**)按全局一致的顺序排列（线性化），并且该排序满足多个操作在时间维度上的实际发生先后顺序。（全局时钟视角下指令的实际执行顺序）
- **顺序一致性：**
  - 任意客户端看到的所有针对分布式系统的**操作**(读、写等原子操作)按全局一致的顺序排列（线性化），同一个客户端发出的多个操作的顺序与该全局一致排序并不矛盾。
  - 满足顺序一致性的指令执行顺序与全局时钟（强一致性的关键点）下看到的顺序可能并不一致。
- **因果一致性：**
  - 不同客户端看到的所有针对分布式系统的操作(读、写等原子操作)排序不一定一致，但该排序不违背操作发生的因果关系。
- **最终一致性：**在分布式系统停止更新时，最终所有读操作都可以获得最新版本的数据。



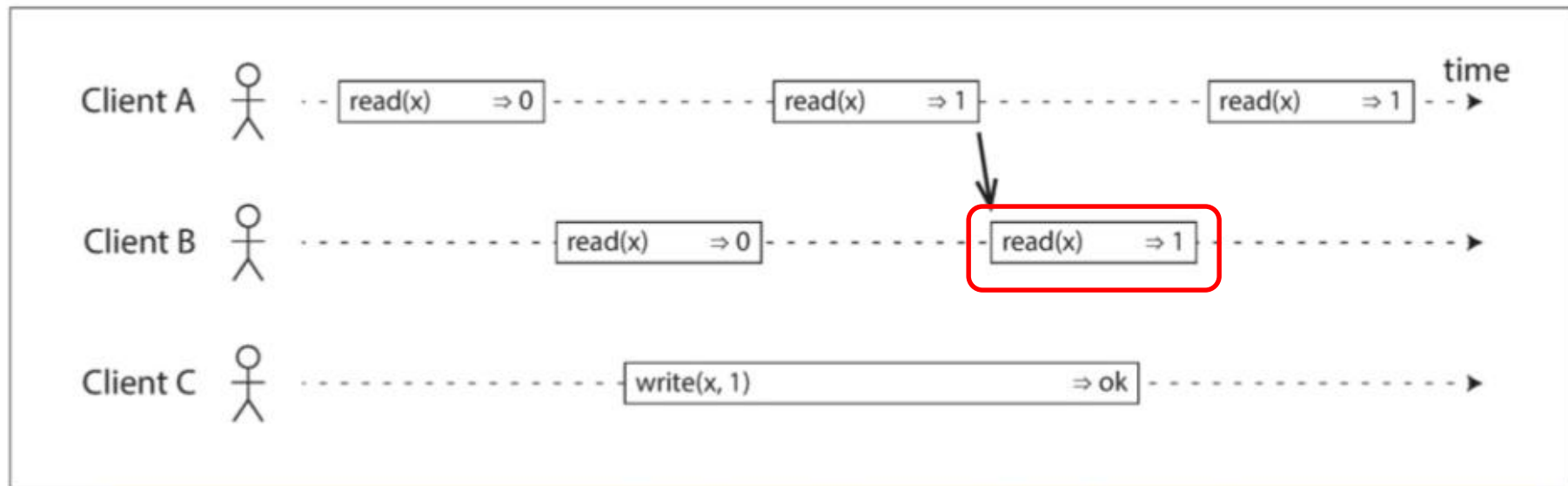
| 一致性模型           | 全局统一顺序 | 保证真实时间顺序 | 保证写后读到新值 | 保证因果关系 | 最终收敛 |
|-----------------|--------|----------|----------|--------|------|
| 线性一致性<br>(强一致性) | √      | √        | √        | √      | √    |
| 顺序一致性           | √      | ×        | ×        | √      | √    |
| 因果一致性           | ×      | ×        | ×        | √      | √    |
| 最终一致性           | ×      | ×        | ×        | ×      | √    |







假定以分布式多副本形式存储一个变量 $x$ ， $x$ 初始值等于0



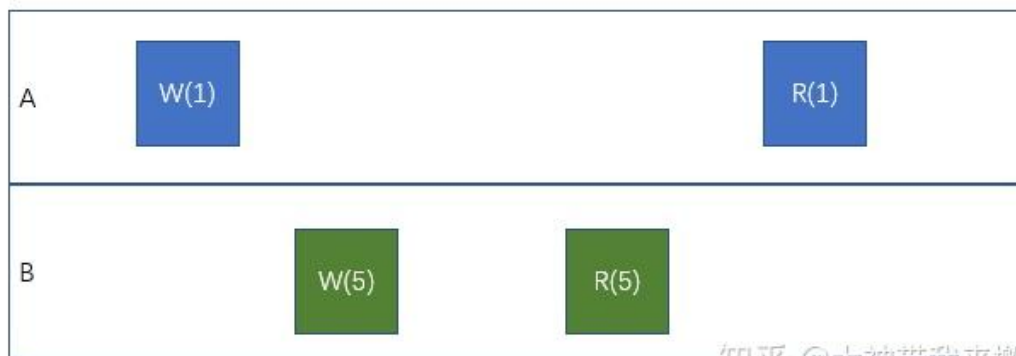
满足强一致性的两个条件：

- (1) 一个写入操作返回成功应答之后，后续的读操作一定可以读出最新的值；
- (2) 一个读操作读取到了一个更新后的变量值，发生在该读操作之后的所有读操作只能读出相同的值或者更新版本的值，不能读出旧版本的值。

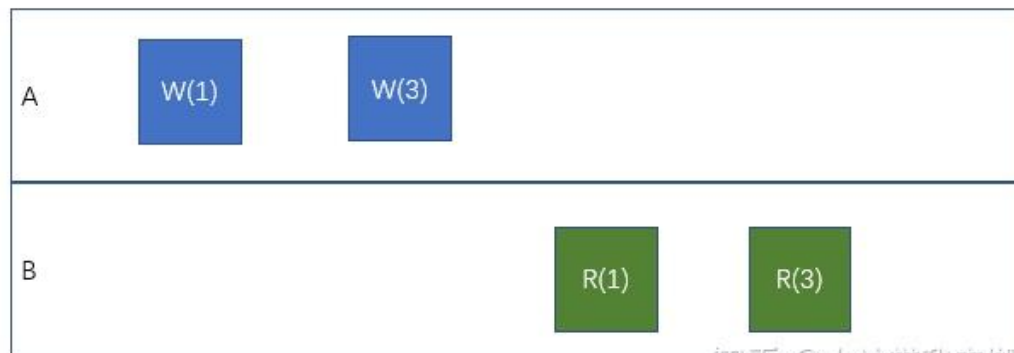
上图中，**Client B**的第二次读操作（红框部分）如果读到的 $x$ 的值为0，则不满足强一致性。（为什么？）



- 顺序一致性只保证单个客户端内部执行指令的顺序，而不关心客户端之间的相对顺序。
- 满足顺序一致性的指令执行顺序与全局时钟（强一致性的关键点）下看到的顺序可能并不一致。

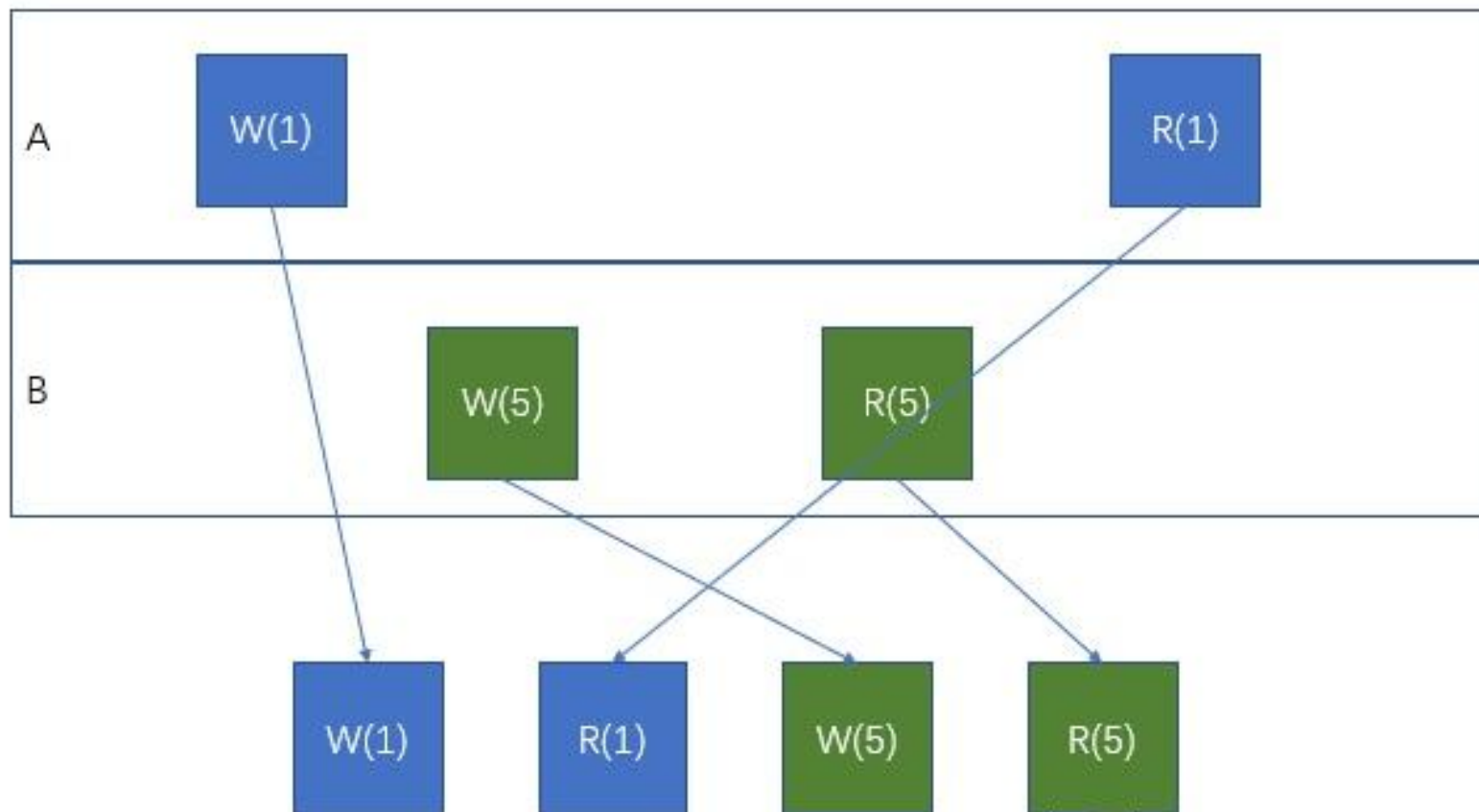


知乎 @大神带我来搬砖

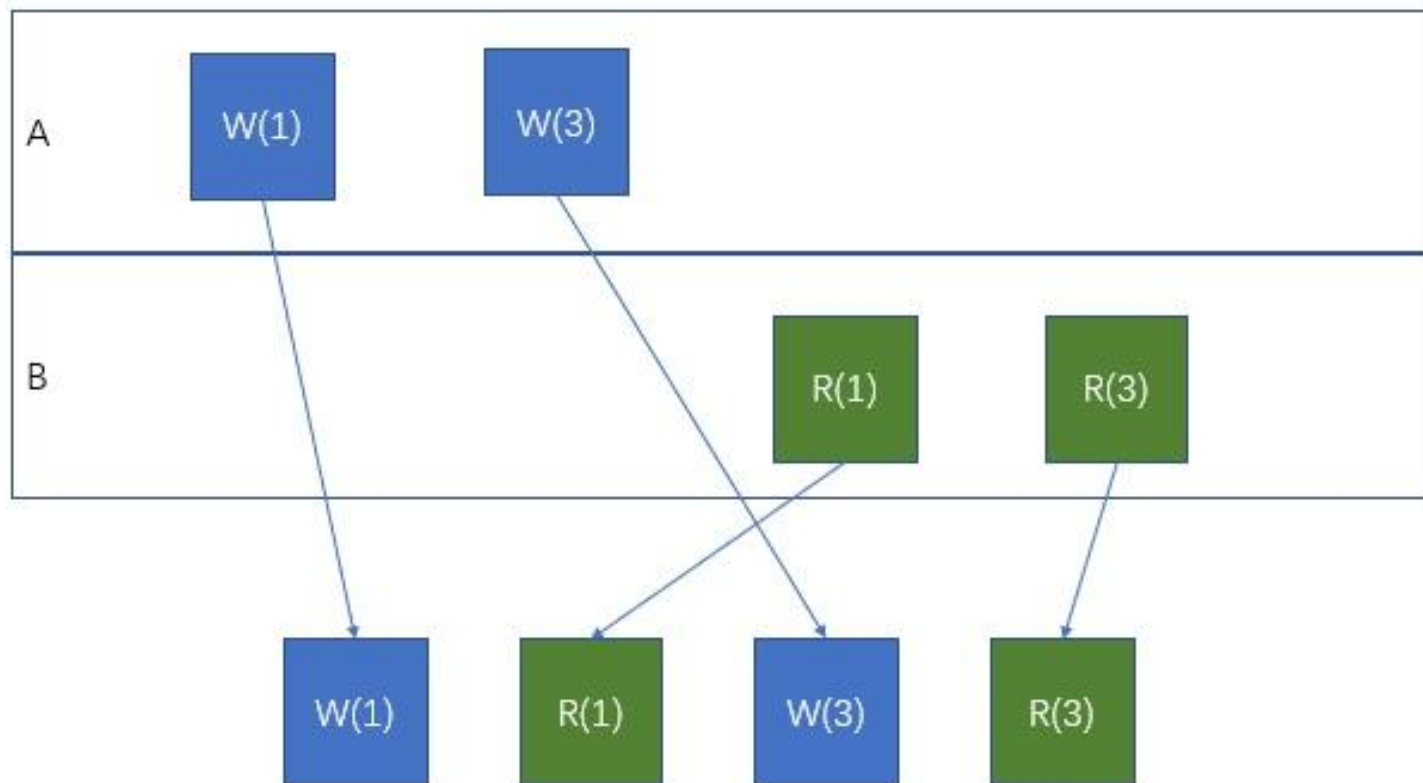


知乎 @大神带我来搬砖

上面的例子都能找到一种全局一致排序满足所有客户端看到的结果。（排序方法见下两页PPT）



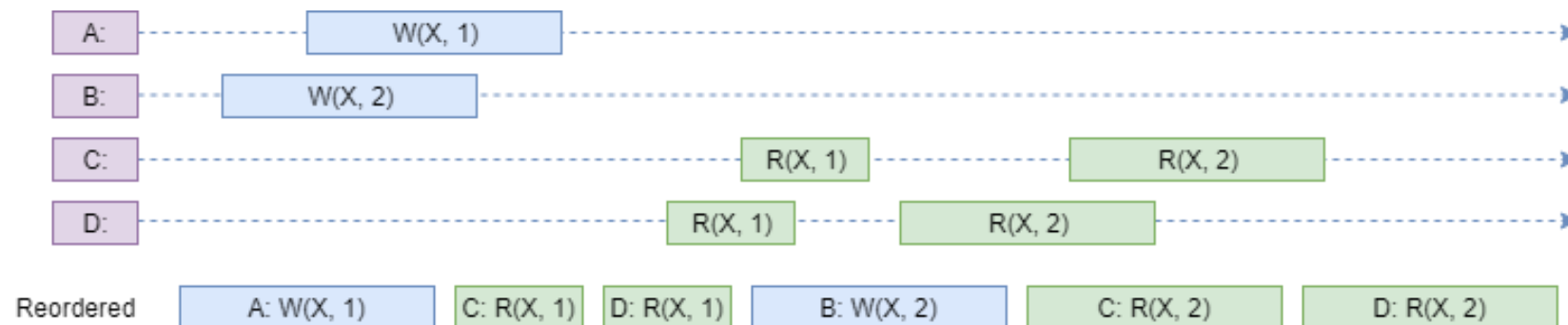
知乎 @大神带我来搬砖



知乎 @大神带我来搬砖



# 顺序一致性的例子





A: --W(x, 1)-----

B: --W(x, 2)-----

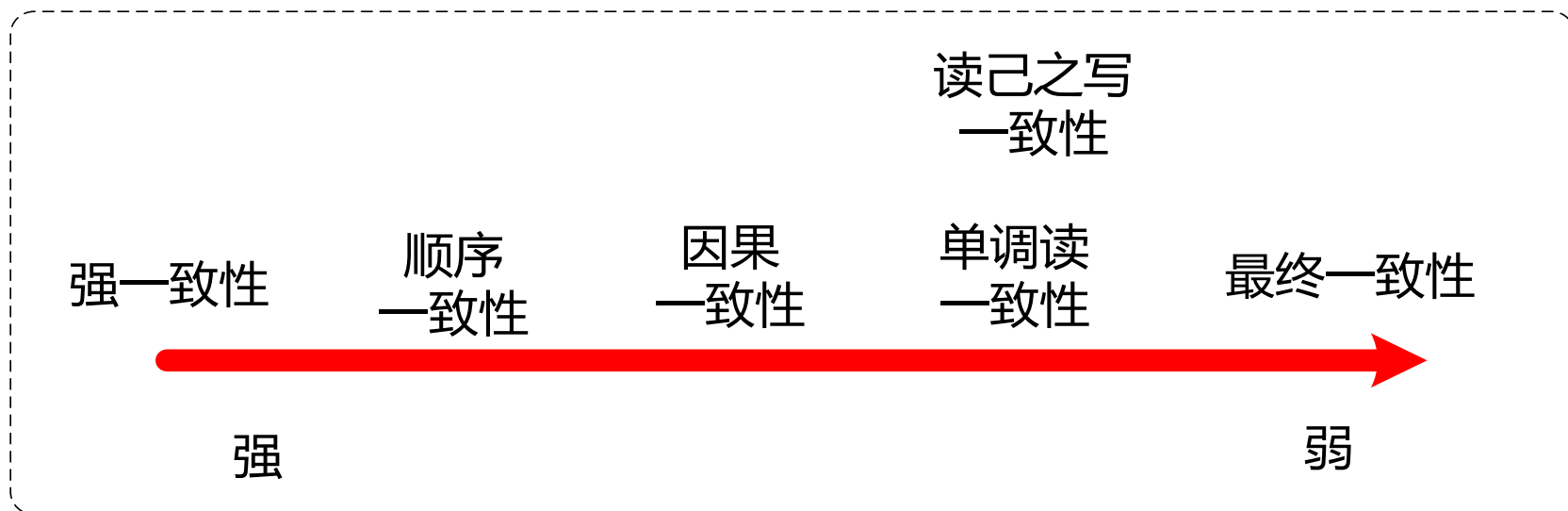
C: --R(x, 2)-----R(x, 1)-----

D: --R(x, 1)-----R(x, 2)-----





- 几乎所有的分布式存储系统都支持最终一致性。
- 分布式系统的一致性保证越强，客户端应用程序越容易编写；一致性保证越弱，客户端程序逻辑就越复杂。
- 一致性保证越强，分布式系统的实现代价就越高，性能就越低。因此可以通过牺牲部分一致性获得更好的性能和容错性。





首页 > 产品 > 云溪 NewSQL Online

## 云溪 NewSQL Online

云溪 NewSQL Online数据库服务是浪潮推出的一款100%自主知识产权的HTAP数据库服务，支持分布式计算和分布式存储，提供多模存储引擎，拥有强一致、无限横向扩展、多中心、高可用等特点并且提供数据库原生自动部署、自动备份、自动容灾、数据恢复、监控等全套解决方案。

立即购买

📍 适用区域：华北三

📖 帮助文档：华北三 >



## 澎湃性能

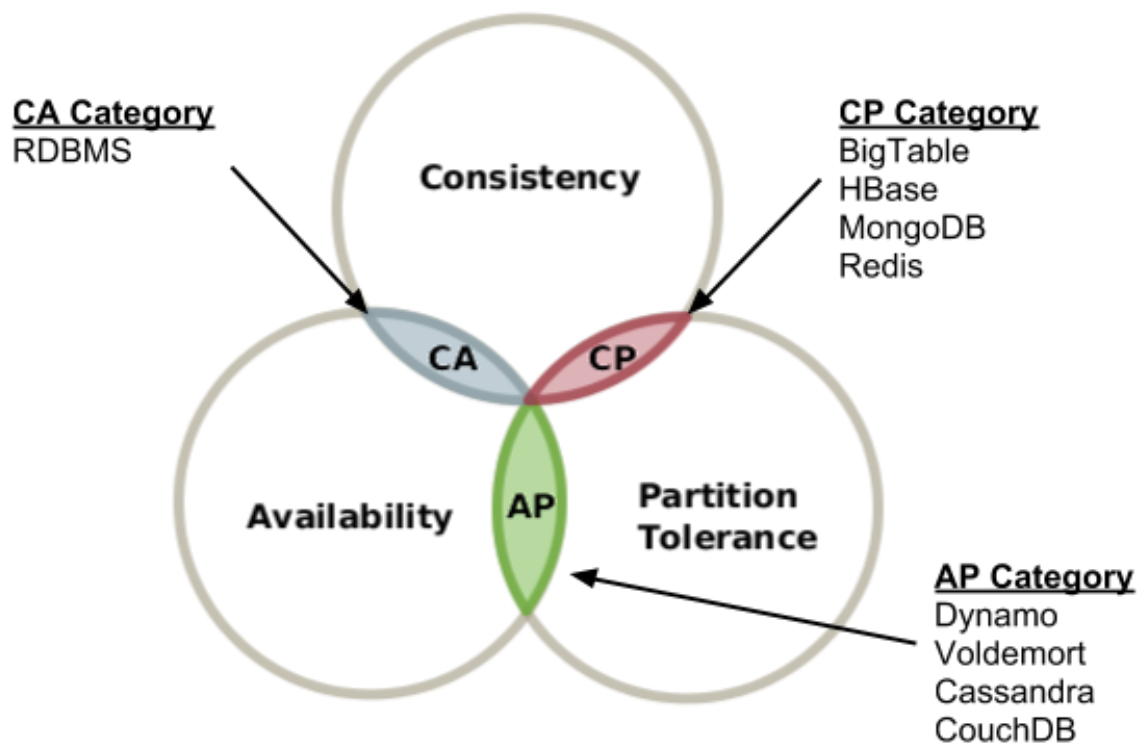
采用计算&存储分离架构，性能最高提升6倍，单节点的场景。

## 稳如磐石，数据安全永远是第一位

PolarDB采用“一主多从”架构，同一实例的所有读写和只读节点都访问存储上同一个数据副本。集群版单机故障，可实现主备切换0数据丢失，彻底解决异步复制带来的主备节点数据非强一致问题，几分钟内即可扩展只读副本，备份和恢复数据。



- Consistency(一致性): 不同节点上数据的强一致性
- Partition Tolerance(割断容忍性): 允许部分节点与其它节点断裂
- Availability(可用性): 发出的请求在规定时间内总能返回结果 (请求响应延时短, 可用性高; 否则可用性低)
- CAP定理: 在设计分布式系统时, 三者只能取其二, 不能三者兼得。





- **BASE理论**是对CAP中一致性和可用性权衡的结果，来源于对大规模互联网分布式系统实践的总结，核心思想是即使无法做到强一致性，但每个应用都可以根据自身业务特点降低部分一致性获得可用性。
- **基本可用** (Basically Available) :
  - 在出现故障的时候，允许损失部分可用性，保证核心可用。
- **软状态** (Soft State)
  - 允许系统中的数据存在中间状态，允许系统在多个不同节点的数据副本存在更新延时。
- **最终一致性** (Eventually Consistent)
  - 不可能一直处于软状态，在一定期限后应当保证所有副本保持数据一致性，从而达到数据的最终一致性

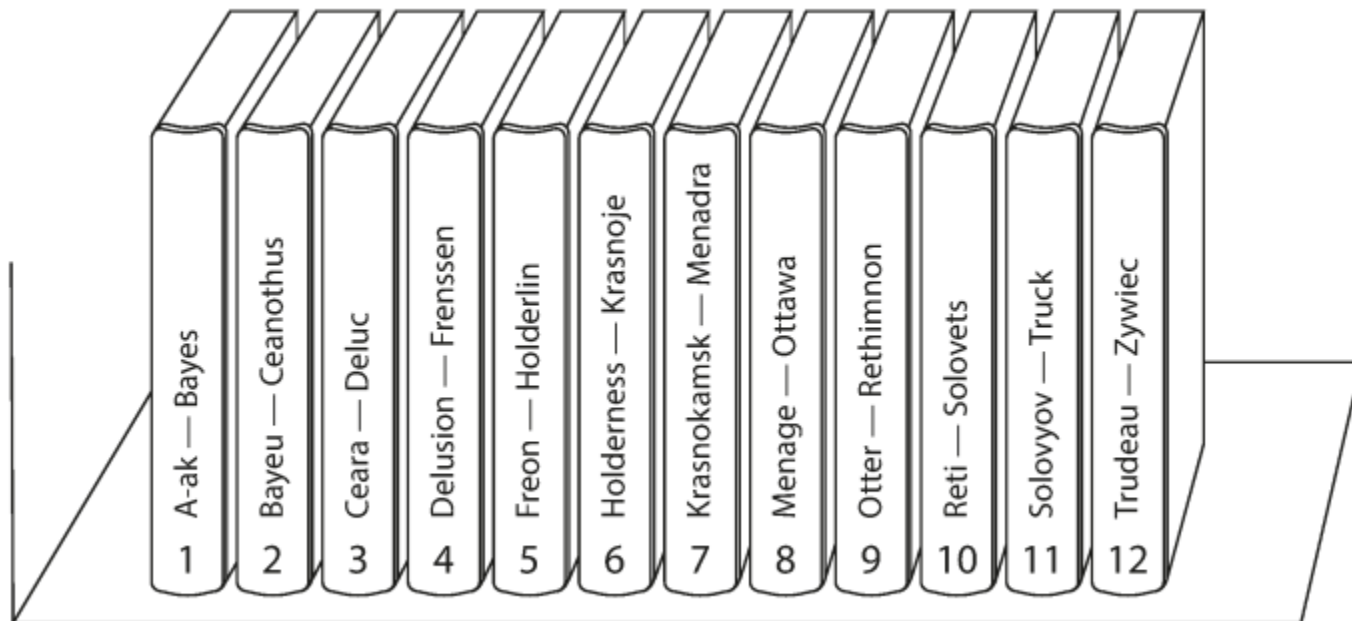


## ■ 合理数据分区的目标:

- 将数据和查询负载**均匀分布**在各个节点上，避免出现**偏斜**(skew)和**热点**(hot spot)问题。
- 分区方式要兼顾跨区查询问题

## ■ 根据主键范围进行分区:

- 数据集合中每个元素（块、对象、记录）都可以找到一个主键，根据主键的连续范围进行分区。
- 各个主键范围分区一般都是**非均匀分布**的。
- 不同的主键范围分区分配给不同的物理存储节点。在特定分区进行分裂或合并时会产生数据移动。
- 优点：按主键进行连续查询很方便。
- 缺点：在主键范围非均匀分布时必须建立**全局索引**以记录数据分区和存储节点的对应关系。一般要专门指定一个节点维护全局索引，该节点是中心节点。

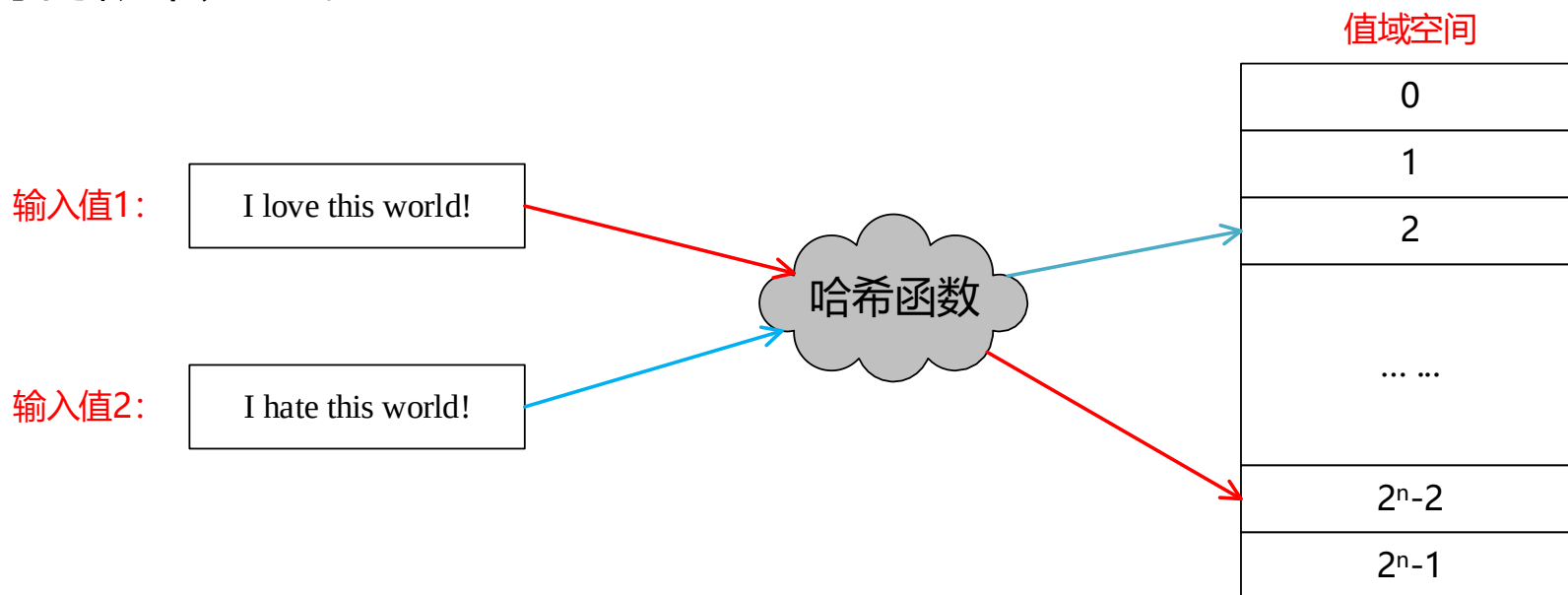


百科全书按主键范围进行分区的例子



## 理想哈希函数的特性：

- ① 确定性：相同的输入会产生相同的输出；
- ② “随机性”：输入输出的对应完全是随机的。对于给定的输入字符串，函数输出“随机”落在值域空间的某个点上。两个输入值即便只相差1个bit，输出值可能相差很远。
- ③ 无碰撞性：假定函数输出n比特的整数，任取两个输入，它们的输出值相等的概率为 $2^{-n}$ 。







## ■ 根据主键的哈希值进行分区：

- ① 将整个哈希空间均匀分成 $k$ 个区间（ $k$ 个桶），每个存储节点负责一个哈希区间（桶）。
- ② 计算新插入数据元素主键的哈希值，然后计算该哈希值落入了哪个桶，最后将该数据元素分配给该桶所对应的存储节点。
- ③ 在增加或删除物理节点时会产生大量数据移动（为什么？）。

## ■ 优点：

- ① 可以在一定程度上避免了偏斜和热点问题。
- ② 不用事先知道主键的取值范围。
- ③ 无须全局索引，因而也无须中心节点。

## ■ 缺点：

- ① 基于主键进行连续范围查询效率极低。
- ② 在物理存储节点较少时仍然会出现偏斜和热点问题。（可以采用虚拟节点的方法进行缓解）
- ③ 桶的个数改变时会产生大量数据移动。



- 一致哈希 是一种特殊的哈希算法。在使用一致哈希算法后，桶数量的改变平均只需要对 $K/n$  个关键字重新映射，其中  $K$ 是关键字的数量， $n$ 是桶的数量。：
- 均衡性：每个桶中映射的关键字数保持均衡
- 单调性：当增加新的桶时，部分关键字被放入新桶中，但不会出现部分关键字在两个旧桶之间转移的情况



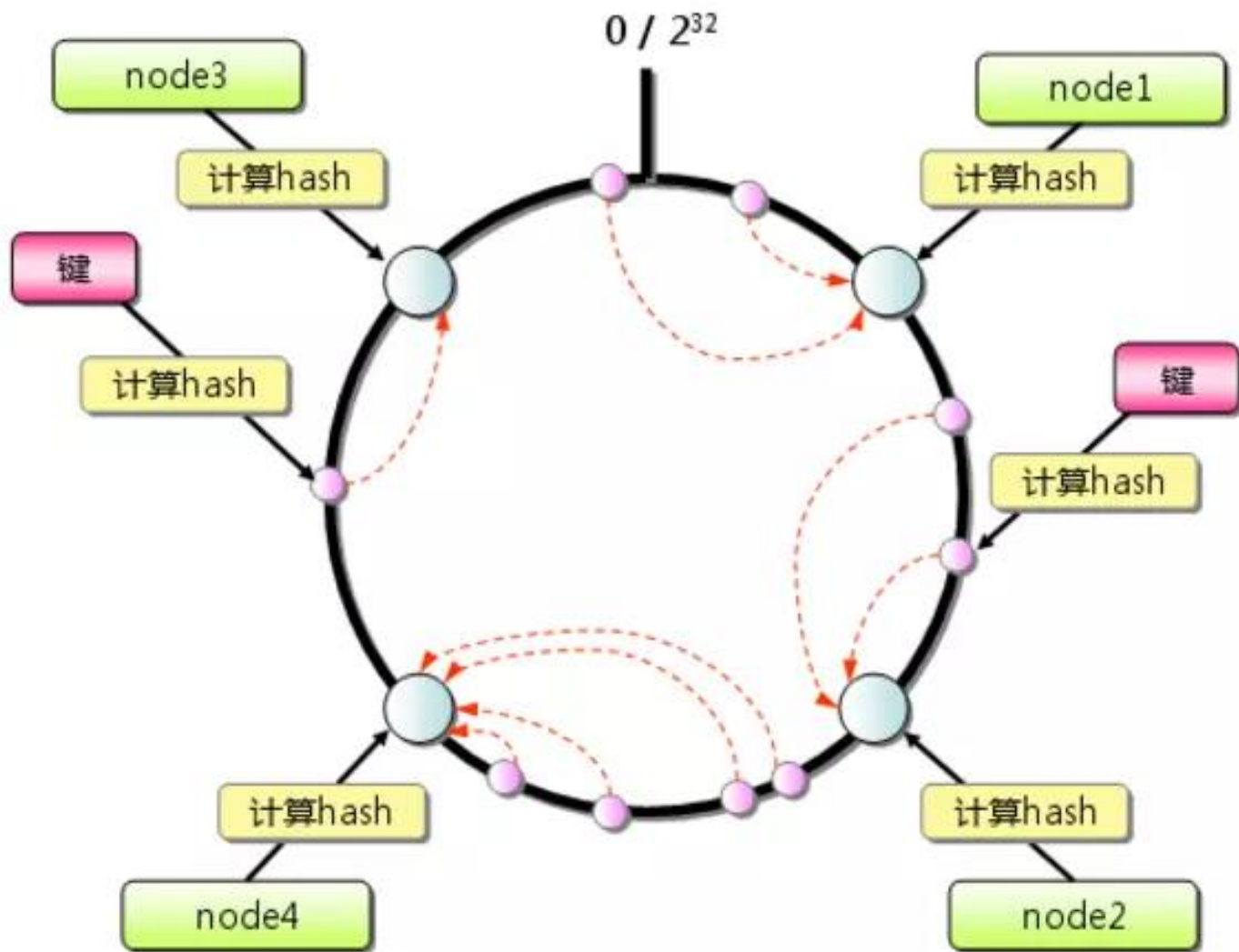
- 找一个普通的理想哈希函数HASH。
- 将该HASH函数的值域空间首尾相接做成一个**环**。
- 假定每个存储节点（每个节点负责一个**桶**）都有一个唯一标识（例如针对每个节点生成一个随机数作为其标识；或用其IP地址）： $ID_1, ID_2, \dots, ID_n$ 。
- 每个节点的唯一标识作为输入计算其HASH函数输出，并将其输出值映射到**环**上，会产生n个**映射点**。
- 节点 $ID_i$ 所负责的桶就是环上从映射点 $HASH(ID_i)$ 开始到下一个映射点之间的连续哈希值区间。（假定**按逆时针旋转**）

思考问题：

- ① 增删节点时多少个桶会受到影响？
- ② 当节点较少时每个节点负责的桶的大小不均衡怎么办？（虚拟节点法）

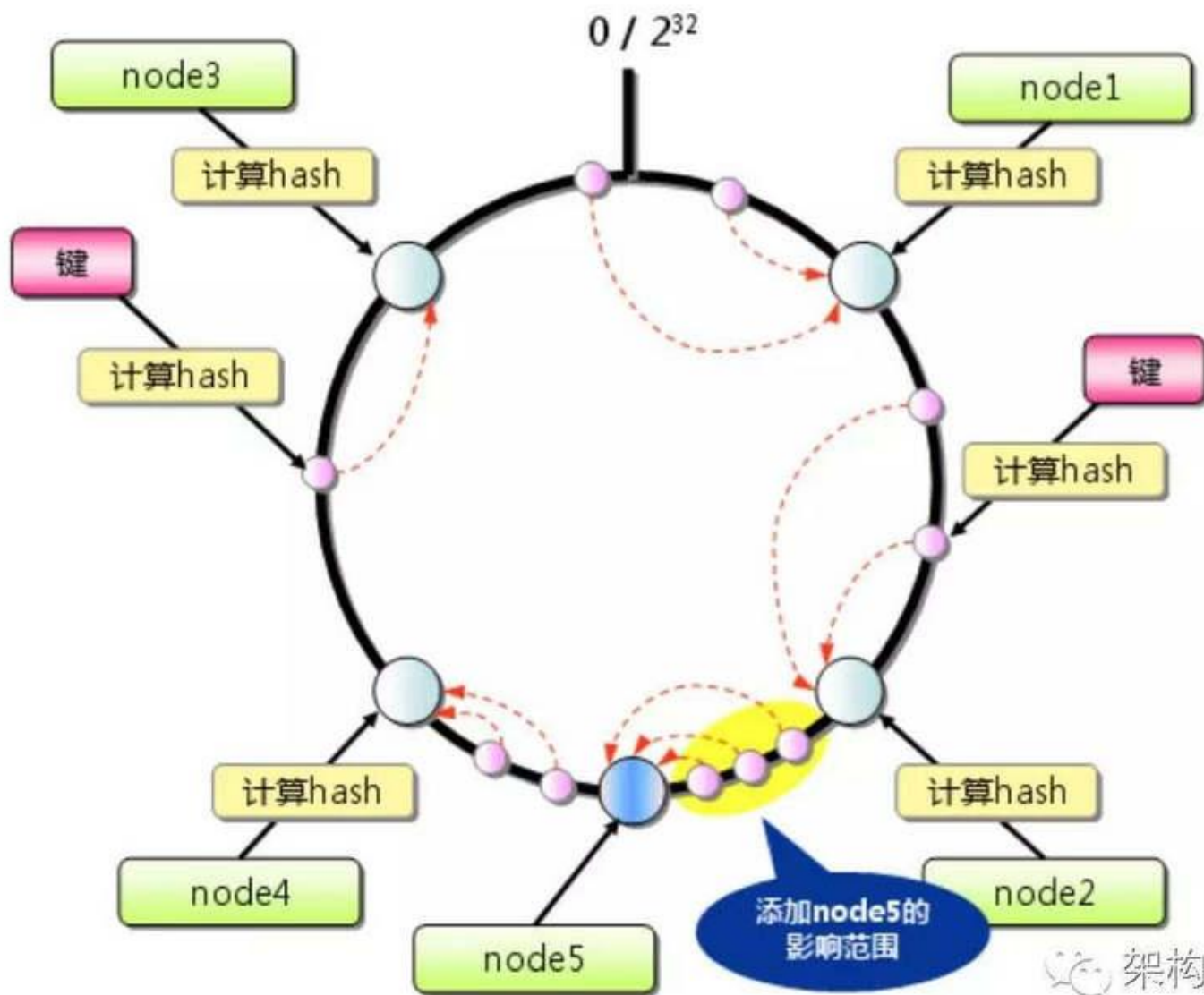


# 基于一致性哈希算法的分区





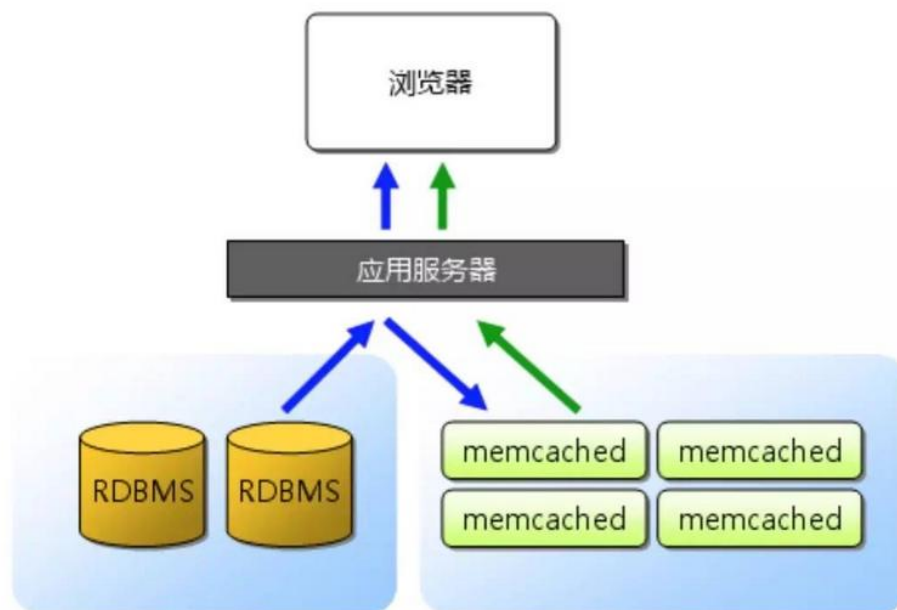
# 增删节点时的分区再平衡





## ■ 什么是MemCached: <https://memcached.org/>

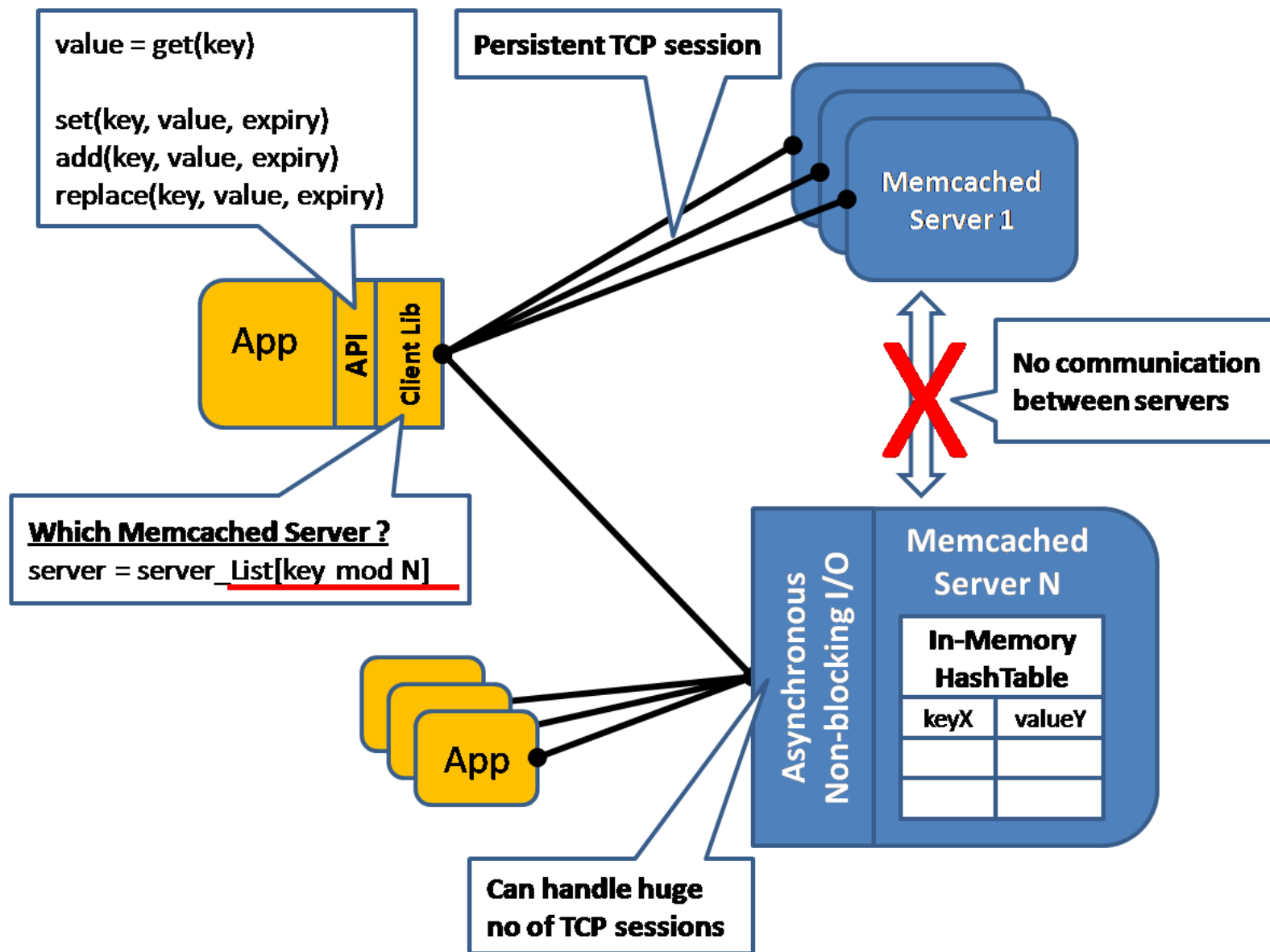
- ① Memcached是一个开源的高性能分布式内存缓存服务器软件。
- ② Memcached支持的是键值对(Key-Value)存储模型，逻辑上相当于将一个巨大的内存HashMap分布存储在多个Memcached节点上。
- ③ 常用于缓存数据库查询结果，减少数据库访问次数，提高动态Web应用的响应速度。



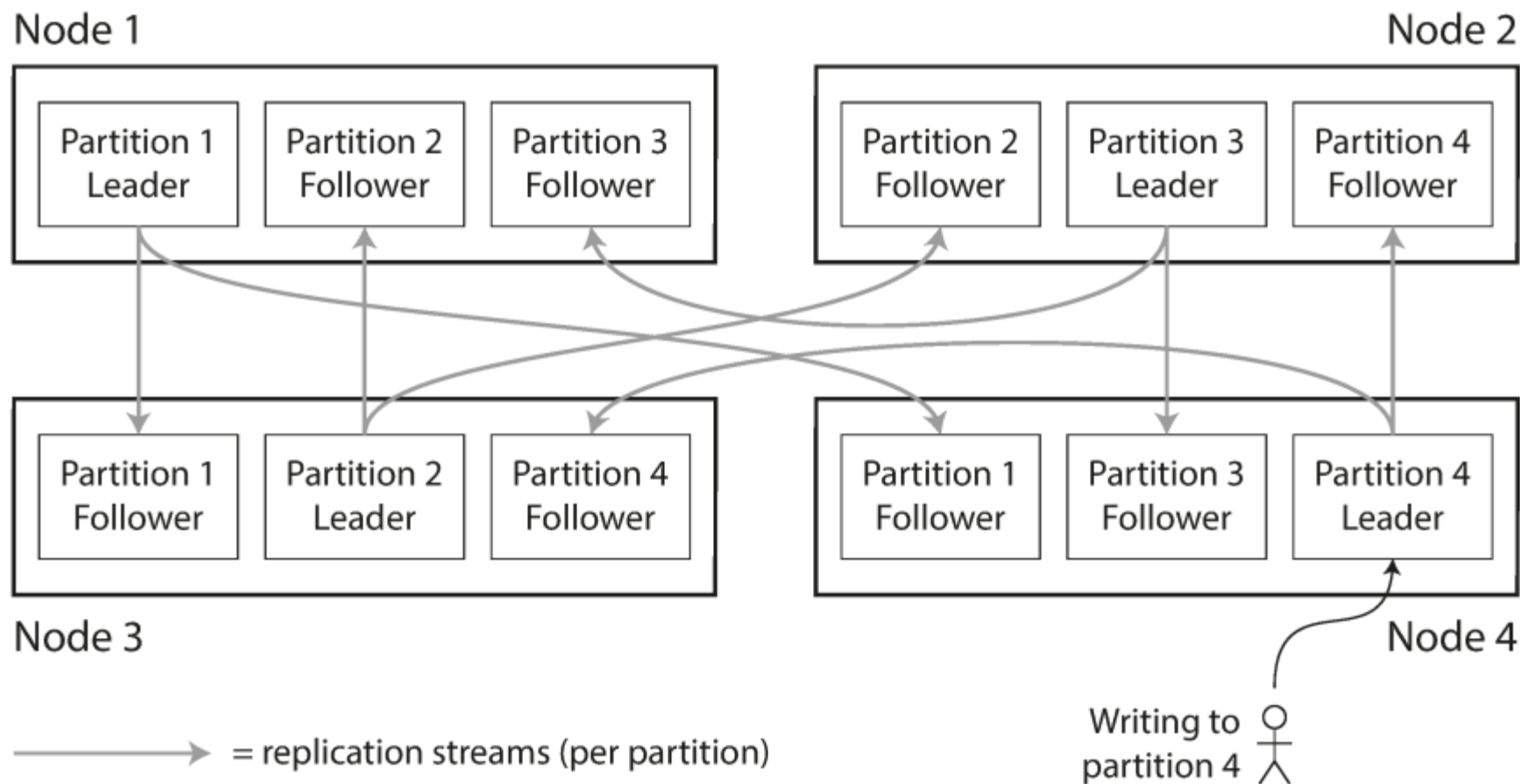
- ➡ 首次访问：从RDBMS中取得数据保存到memcached
- ➡ 第二次后：从memcached中取得数据显示页面



# 数据分区案例：MemCached

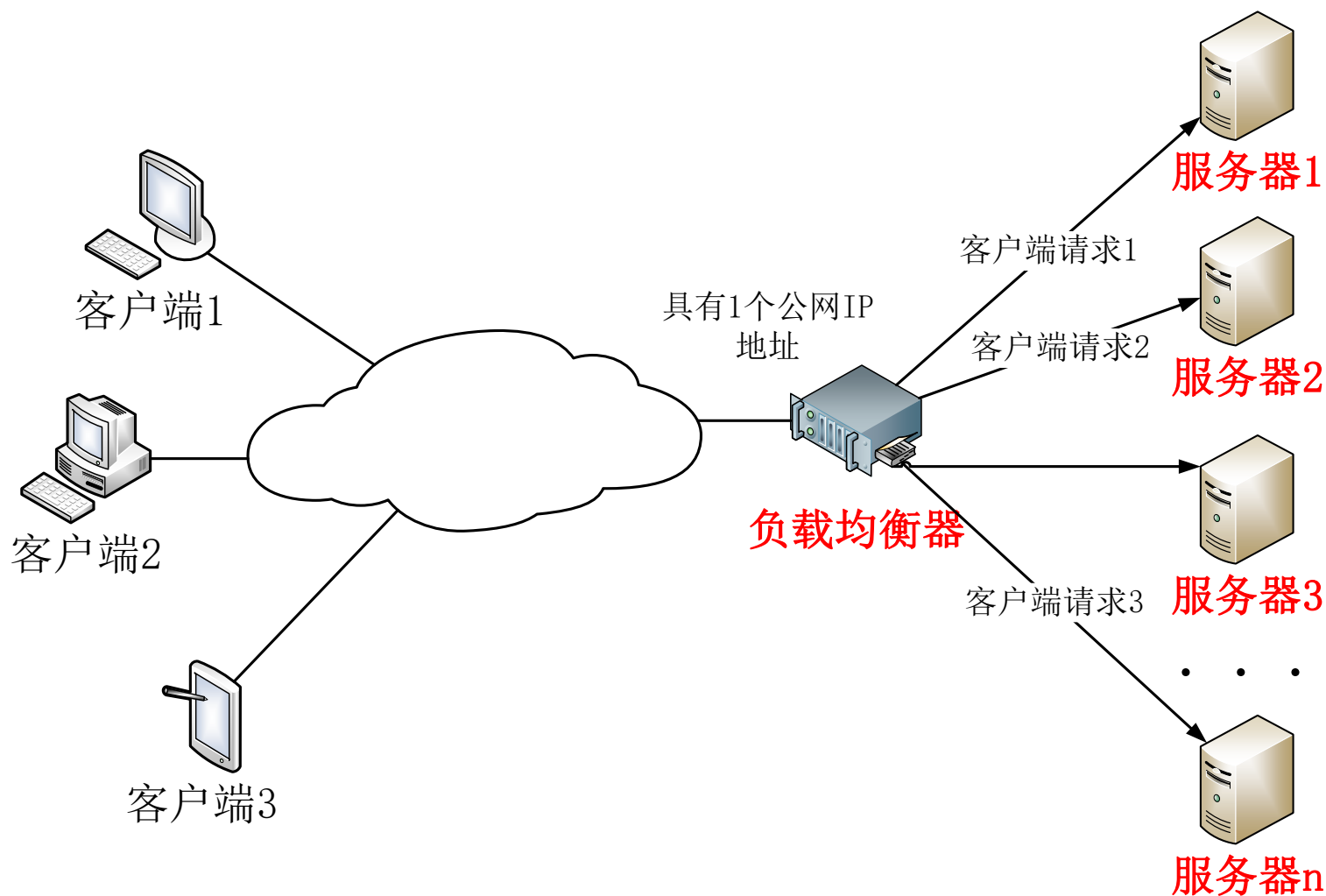






组合使用复制和分区：每个节点充当某些分区的领导者，其他分区充当追随者



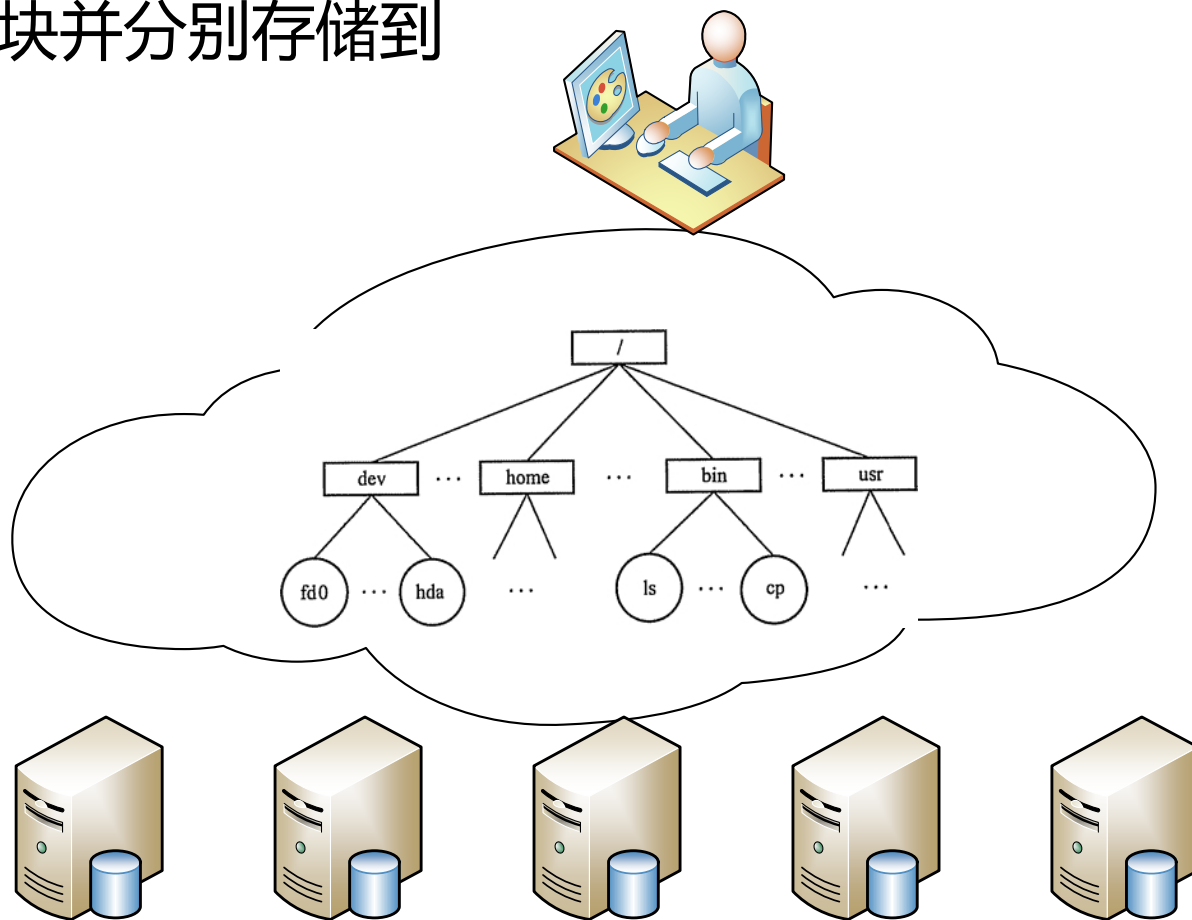




- 随机
- 轮询
- 固定权重值
- IP哈希（基于一致性随机散列函数）
- 最少TCP连接数
- 最小响应时间
- 基于各服务器实际负载的动态负载均衡算法



- 分布式文件系统：将分布式系统中多个节点的存储资源整合在一起，向用户/应用程序呈现统一的存储空间和文件系统目录树。用户无需关心数据存储在哪个节点上。
- 大文件被自动分块并分别存储到不同的节点上。





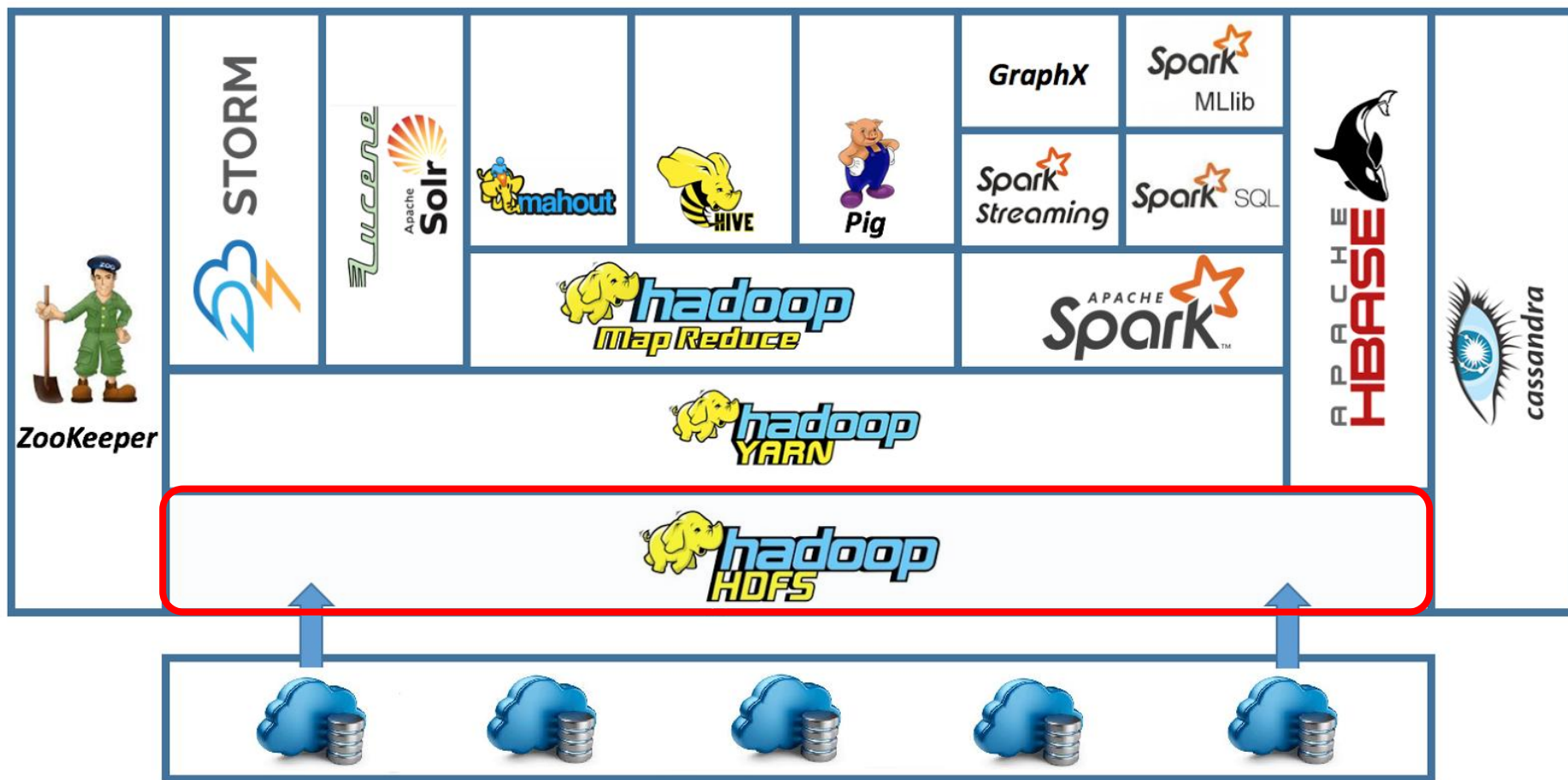
- 由分布式文件系统中间件自动处理文件的分块、存储节点选择和副本备份等问题。
- 统一名字空间：每个文件、目录在分布式文件系统中都有统一的、唯一的名字。
- 锁管理机制：对多个用户的并发读写具有锁控制措施。
- 副本备份机制：一般都支持利用多副本提高数据容错性。
- 数据存取方式：随机写入或追加式写入
- 安全机制：对读写文件有权限控制措施
- 可扩展性：支持通过增加节点扩充存储容量
- 高吞吐率：多副本可以提高数据吞吐率



- GFS: Google File System, Google公司内部所用, 见论文《The Google File System》
- HDFS: Hadoop Distributed File System, Apache根据GFS论文实现的开源分布式文件系统, Hadoop生态系统的重要组成部分
- Ceph: 加州大学的Sage开发的开源分布式文件系统, 被云平台广泛应用, 已被集成到Linux内核。同时支持块存储、对象存储和文件存储。
- TFS: Taobao File System
- FastDFS
- GridFS
- MogileFS



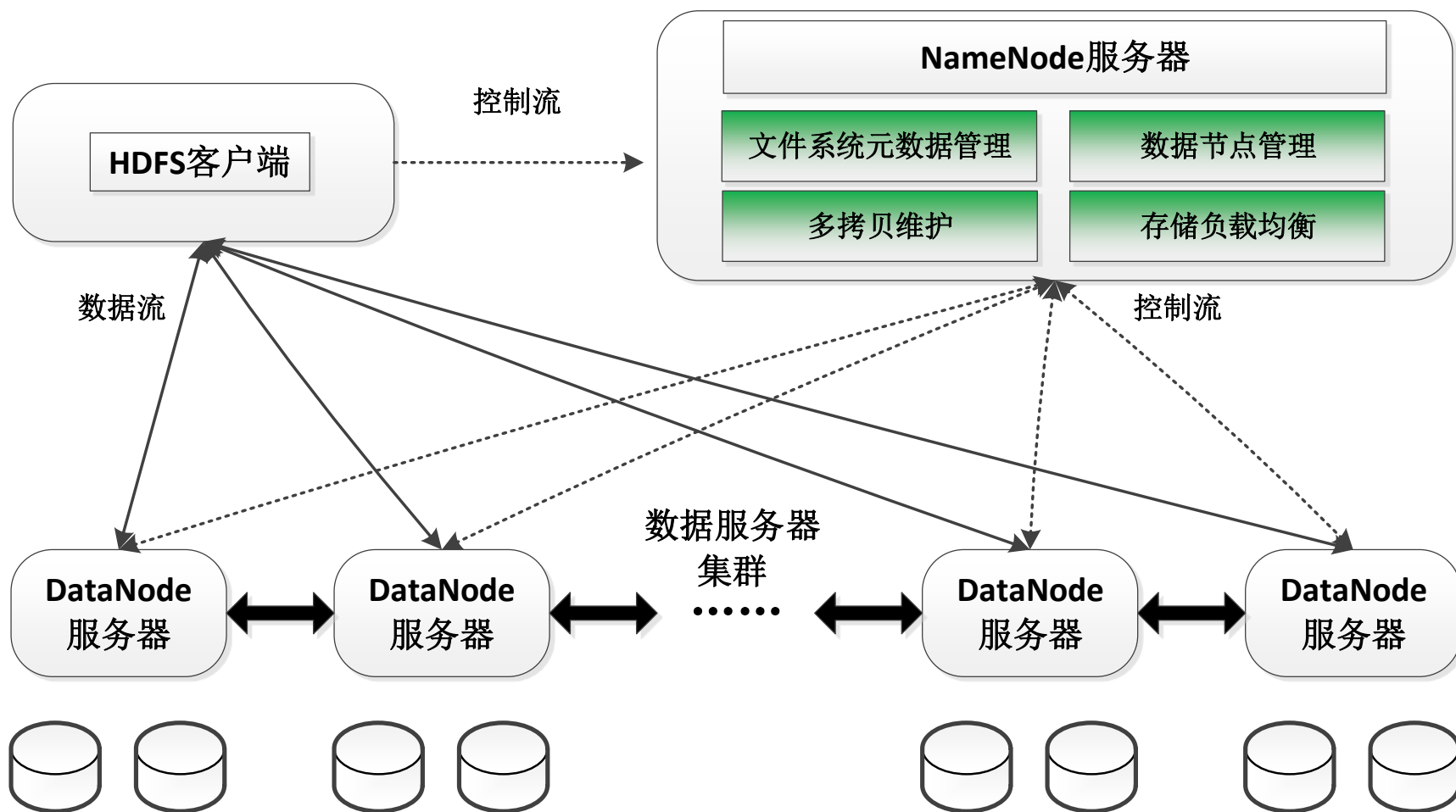
- HDFS 遵循主/从架构，由单个 NameNode(NN) 和多个 DataNode(DN) 组成。
- 高容错：采用数据多副本方案，部分硬件的损坏不会导致全部数据的丢失。
- 高吞吐量：支持多个客户端同时与多数据节点直接通信，实现了高吞吐量的数据访问。（不同于低延迟数据访问）
- 大文件支持：HDFS 适合于大文件的存储，文档的大小应该是 GB 到 TB 级别的。（为什么不适合大量小文件存储？）
- 简单一致性模型：HDFS 更适合于一次写入多次读取 (write-once-read-many) 的访问模型。支持将内容追加到文件末尾，不能从文件任意位置新增数据。



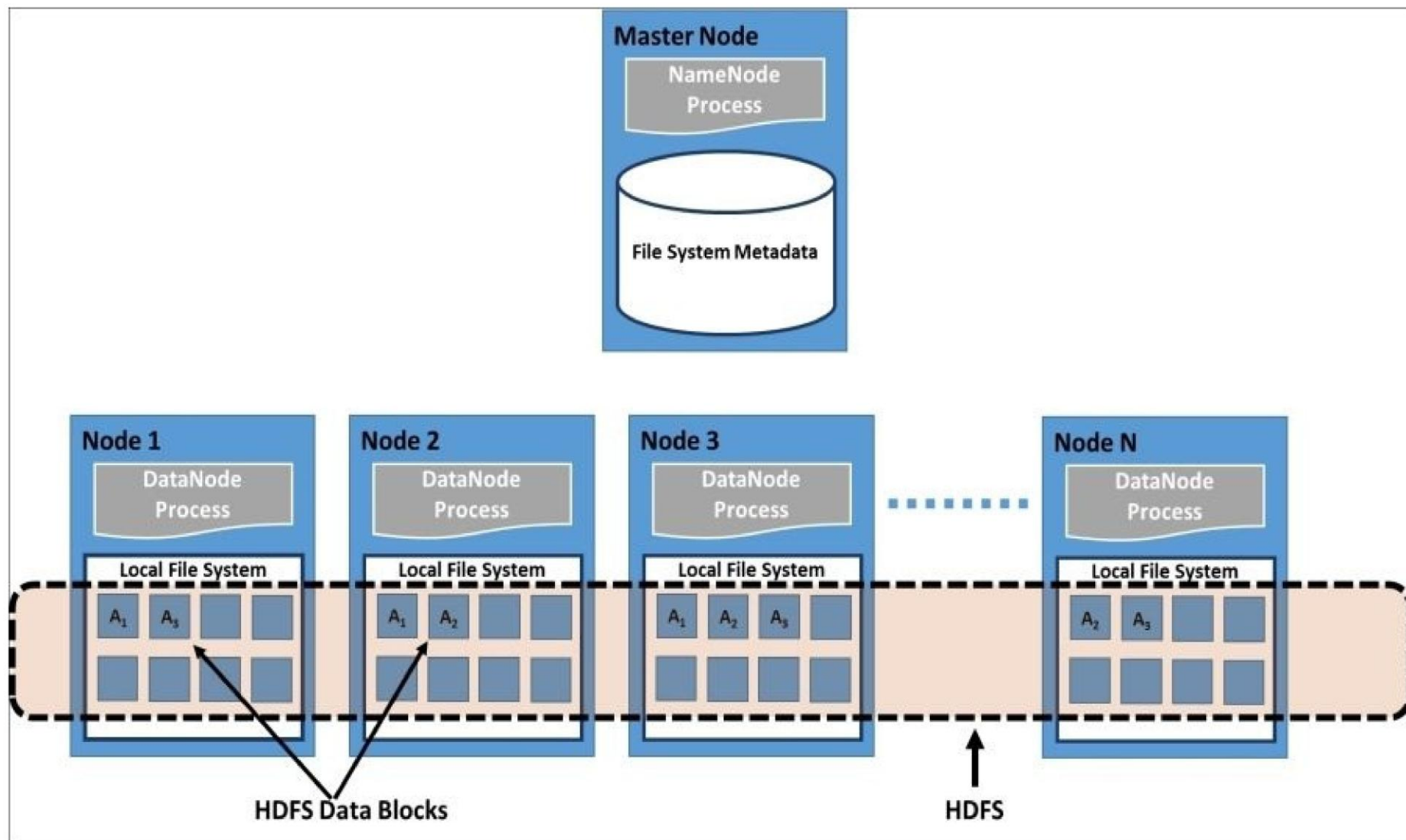
- Hadoop平台：面向大数据的开源分布式平台，包含了收集、存储、批处理、实时处理、分析和可视化大数据的一系列软件系统。



- 与MapReduce分布式计算框架紧密集合
- 一次写入，多次读取









注：HDFS中的每个数据块都有全局唯一的编号

**Table1: 文件名—数据块对应表**

| 目录文件名             | 数据库块编号列表 |
|-------------------|----------|
| /user/alice/file1 | 1, 2, 5  |
| /user/bob/file2   | 3, 4, 6  |

**Table2: 数据块—物理节点对应表**

| 数据块编号 | 存储该数据块节点的IP地址 |
|-------|---------------|
| 1     | DN1, DN2      |
| 2     | DN2, DN3      |
| 3     | DN1, DN3      |
| ...   | ...           |



## NameNode

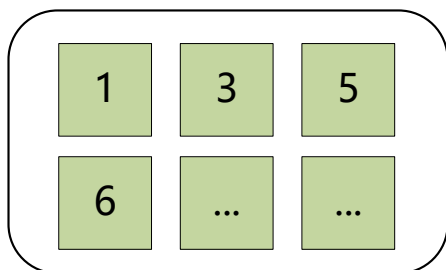
### Metadata

|                   |      |       |
|-------------------|------|-------|
| /user/alice/file1 | ---> | 1,2,5 |
| /user/bob/file2   | ---> | 3,4,6 |

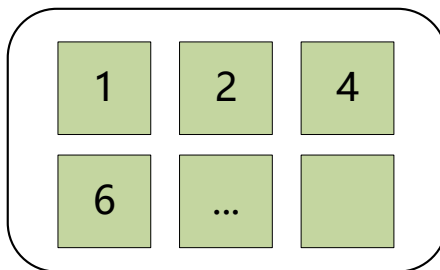
|     |      |          |
|-----|------|----------|
| 1   | ---> | DN1, DN2 |
| 2   | ---> | DN2, DN3 |
| 3   | ---> | DN1, DN3 |
| ... | ---> | ... ..   |

文件块备份数为2

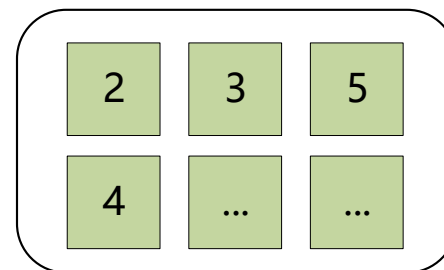
## DataNode1

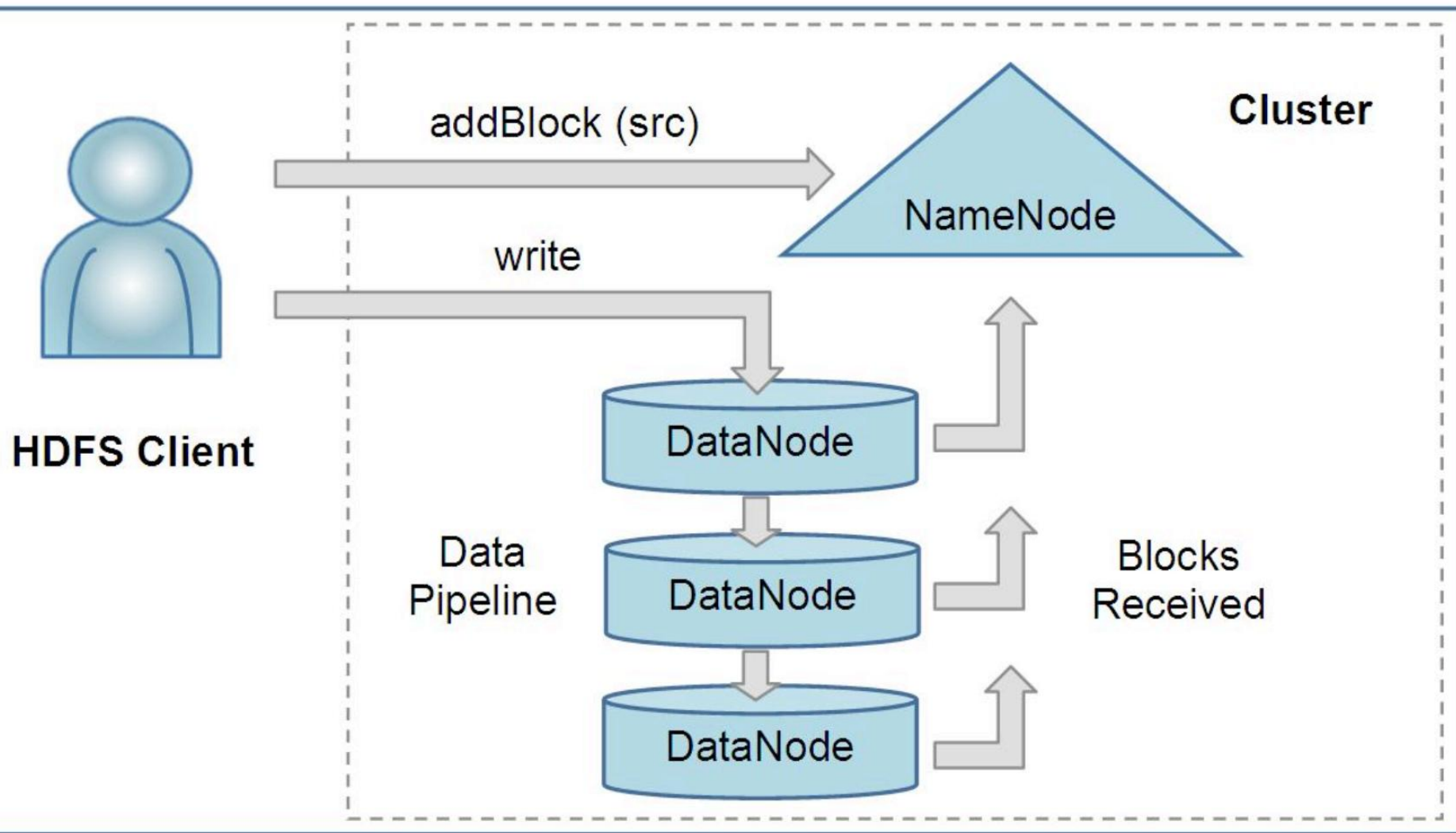


## DataNode1



## DataNode1







## ■ HDFS读数据流程：

- ① 客户端将读请求发送给NameNode，读请求参数中包含了文件名、偏移量和长度。
- ② NameNode根据文件名、偏移量查找自己内存中“文件名—数据块对应表”和“数据块—物理节点对应表”，并将包含目标数据块的数据节点IP列表发送给客户端。
- ③ 客户端从数据节点IP列表中选择“最近的”的数据节点，并与该节点建立Socket连接 直接读取数据。

## ■ HDFS写数据流程：

- ① 客户端将新建文件请求发送给NameNode，NameNode根据负载均衡策略选择3个数据节点，并将这些节点IP地址返回给客户端。
- ② 客户端将这3个数据节点构成一个流水线，将第一个数据库的数据流写入流水线。
- ③ 第一个数据块写入成功后客户端再向NameNode获取下一个数据块对应的3个数据节点。